

Technische Architektur

Sanitisierte Modullandkarte und öffentliche Claim-Grenzen.

Erstellt 2026-06-17

Architekturprinzip

Der wiederkehrende Gedanke ist Trennung von Autorität. Eine WhatsApp-Nachricht, ein Sprachbefehl oder ein lokales Event wird nicht direkt zu beliebiger Ausführung, sondern erst interpretiert, normalisiert, geroutet, über Tools ausgeführt, bewertet und dann zurückgemeldet.

Sanitisierte Modullandkarte

Schicht	Was ich zeigen kann	Öffentlich sichere Evidenz
Remote/Dual-Brain	Mobile Anfragen werden zu strukturierten Arbeitsaufträgen.	Workflow-Notizen, Watcher, MiguelAgent, WA Task Manager.
Miguel Core	Lokale Kontrollschicht für Routen, State, Modelle, Memory, Tools und Orchestrierung.	Daemon-Routen, Orchestra Worker, Handover-Dokumente.
Model Cascade	Routing nach Kosten, Qualität, Quoten, Cooldowns und Fehlern.	AI Router, Cascade, LiteLLM-Konfiguration.
Memory	Raw/Working/Procedural Memory mit Widerspruch und Staleness.	CASS, Memory Broker, Vault Watcher, Smart Memory MCP.
Voice	Sprache als sicherheitsrelevante Tool-Oberfläche.	Voice Source, Pipeline Notes, Shell Guard.
ANIMA	Spekulative Kognition als Code, Tests und Evidenzgrenze.	Kernel, Tests, Benchmarks, Methodik.

Öffentliche Claim-Grenzen

Arbeitskoerper	Sicherer Status	Grenze
Remote Agent Workflow	gebaut/entwickelter Prototyp	Keine produktionsgehärtete Sicherheit behaupten.
Miguel Core	lokale Kontrollschicht gebaut	Nicht alle Endpoints als produktionsreif darstellen.
Multi-Modell-Routing	gebaut	Exakte Kostenreduktion nur mit frischen Messungen behaupten.
Memory/Vault/CASS	gebaut/entworfen	Kein menschenähnliches Bewusstsein behaupten.
Voice-System	gebaut plus erweitert geplant	Implementiertes Terminal-Voice und geplante Pipeline trennen.
ANIMA	gebaut/getestet	Kein Bewusstseinsbeweis.
ECSA AMR	Architektur/Zukunftsprojekt	Keine klinische Validierung behaupten.
AMR-HORIZONT / FFG	Projektpaket vorbereitet	Keine bewilligte Foerderung behaupten ohne Nachweis.
IP/PCT	formale technische Ausarbeitung	Keine erteilten Rechte behaupten ohne Verifikation.
Quantum/EXON	spekulativ/red-teamed	Kein FTL- oder Physikdurchbruch.

Warum ich keine rohen Screenshots als Beweis nutze

Screenshots zeigen Oberfläche, aber keinen Mechanismus. Diese Mappe zeigt deshalb Architektur, Modulverantwortung, Ablauf und Claim-Grenzen.

Rohe Quellordner bleiben bewusst draussen, weil sie private Pfade, Zugangsdaten, persönliche Notizen, IP-Material und unfertige Experimente enthalten können.

Remote WhatsApp-to-Agent Command Loop

Warum ich es gebaut habe: Ich wollte von unterwegs mit meinem Handy Arbeit an meinen Laptop übergeben können, ohne daraus eine gefährliche direkte Shell zu machen. Die WhatsApp-Nachricht sollte erst verstanden, in einen Arbeitsauftrag übersetzt, ausgeführt, bewertet und dann sauber zurückgemeldet werden.

Grenze: Persönlicher Prototyp bzw. entwickeltes System, keine Behauptung eines produktionsgehärteten Sicherheitsprodukts.

Remote WhatsApp-to-Agent Command Loop - Architektur

Ich zeige die Architektur bewusst als Ablauf, weil der Wert in der kontrollierten Sequenz liegt.

- WhatsApp oder Audio kommt in der OpenClaw-Schicht an.
- Ein Watcher normalisiert Ereignisse, dedupliziert Nachrichten und behandelt Medien.
- MiguelAgent macht daraus Ziel, Komplexität, Arbeitsauftrag, Erfolgskriterien und Modell-Hinweis.

- Die Ausführungsseite nutzt Miguel/EXOVIUM-Tools und Modellrouting.
- Eine Qualitätsschleife prüft, ob das Ergebnis wirklich fertig ist.
- Erst danach wird die Antwort wieder an WhatsApp geschickt.

Remote WhatsApp-to-Agent Command Loop - was ich zeigen kann

- Ich habe Fernzugriff als Autoritäts- und Sicherheitsproblem behandelt, nicht nur als Komfortfunktion.
- Ich habe die Front-Agenten-Schicht von der tool-reichen Ausführungsschicht getrennt.
- Ich habe mobile Sprache/Chat in strukturierte Arbeit mit Erfolgskriterien übersetzt.
- Schnell bestätigen, langsam und kontrolliert arbeiten, erst nach Bewertung antworten.
- Remote-Steuerung braucht Deduplikation, Unterbrechbarkeit, Fortschritt und Audit-Spuren.
- Private Kanal-IDs und Zugangsdaten gehören nie in ein öffentliches Portfolio.

Remote WhatsApp-to-Agent Command Loop - nächste Härtung

Die öffentliche Version bleibt kuratiert und sicher: keine privaten Pfade, keine Zugangsdaten, keine rohen Arbeitsordner.

- Öffentlichen Demo-Modus mit synthetischen Daten bauen.
- Befehle nach Risiko und Bestätigungsbedarf klassifizieren.
- Tool-Aufrufe, Modellwahl und Bewertung als sichere Audit-Events speichern.

Miguel Core Unified Daemon

Warum ich es gebaut habe: Ich wollte meine vielen einzelnen Agenten-, Voice-, Memory- und Tool-Skripte nicht als lose Sammlung behalten. Miguel Core war mein Versuch, daraus eine beobachtbare lokale Kontrollschicht zu machen.

Grenze: Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.

Miguel Core Unified Daemon - Architektur

Ich zeige die Architektur bewusst als Ablauf, weil der Wert in der kontrollierten Sequenz liegt.

- Ein Python-Daemon bundelt Chat, Streaming, Voice, State, Vault, Knowledge und Tools.
- Routen verbinden Memory Broker, CASS, Vault Watcher, Prediction, Metacognition und macOS Bridge.
- Orchestra delegiert Aufgaben an Worker und verfolgt Status, Timeout, Ergebnis und Abbruch.
- Start- und Handover-Dateien dokumentieren Modellrouting, Boot Checks und Autonomie-Ringe.

Miguel Core Unified Daemon - was ich zeigen kann

- Ich denke Agenten nicht als einzelne Prompts, sondern als Betriebssystem-nahe Services.
- Ich kann verstreute Prototypen in eine zentrale Kontrollfläche zusammenführen.
- Ich habe frueh über Health, State, Tool-Oberflächen und Autonomiegrenzen nachgedacht.

- Eine Route zu haben ist nicht dasselbe wie eine Route produktionsreif zu härten.
- Lokale Systeme brauchen trotzdem Beobachtbarkeit, klare Rechte und Fehlergrenzen.
- Autonomie-Ringe sind eine hilfreiche Sprache für sichere und unsichere Aktionen.

Miguel Core Unified Daemon - nächste Härtung

Die öffentliche Version bleibt kuratiert und sicher: keine privaten Pfade, keine Zugangsdaten, keine rohen Arbeitsordner.

- Stabile Public-API aus dem breiten internen Daemon herauslösen.
- Endpoint-Rechte und Risiko-Labels maschinenlesbar dokumentieren.
- Health Reports für alle Hauptmodule erzeugen.

Multi-Model Cascade Router

Warum ich es gebaut habe: Ich wollte nicht alles blind durch ein Modell schicken. Der Router behandelt Modelle als Ressourcen mit Stärken, Kosten, Quoten, Ausfällen, Qualität und Datenschutzkontext.

Grenze: Exakte Kostenersparnis nur dann behaupten, wenn neue Messlogs sie belegen.

Multi-Model Cascade Router - Architektur

Ich zeige die Architektur bewusst als Ablauf, weil der Wert in der kontrollierten Sequenz liegt.

- Task-Strategien wählen schnell, hochwertig, gratis, lokal, Code, kreativ oder bulk.
- Lokale Modelle und kostenlose APIs werden dort bevorzugt, wo es sinnvoll ist.
- Quoten, Cooldowns und Ausfälle blockieren schlechte Routen.
- Qualitätsgates entscheiden, ob eskaliert wird.
- Mehrere freie Modelle können gegeneinander laufen, langsamere Calls werden abgebrochen.

Multi-Model Cascade Router - was ich zeigen kann

- Ich habe Modellwahl als Governance-System behandelt, nicht als Geschmacksfrage.
- Ich habe Kosten, Latenz, Quoten, Qualität und Fallbacks im selben System gedacht.
- Ich habe lokale und API-basierte Modelle in einer Routing-Logik verbunden.
- Ein guter Agent muss elegant schlechter werden können, wenn Modelle ausfallen.
- Qualitätsgates sollten Eskalation auslösen, nicht Modell-Prestige.
- Outcome-Logging ist der Anfang empirischer Modell-Governance.

Multi-Model Cascade Router - nächste Härtung

Die öffentliche Version bleibt kuratiert und sicher: keine privaten Pfade, keine Zugangsdaten, keine rohen Arbeitsordner.

- Frische Benchmarks über repräsentative Aufgabenklassen laufen lassen.

- Aggregierte Kosten/Latenz/Qualität ohne Geheimnisse veröffentlichen.
- Policy-Labels für privat, öffentlich, billig, schnell und hochriskant einführen.

Memory, Vault, CASS und Smart Memory MCP

Warum ich es gebaut habe: Ich wollte Agenten nicht nur größere Kontextfenster geben, sondern echtes Arbeitsgedächtnis: roh, zusammengefasst, prozedural, widerspruchsbewusst, veraltetheitsbewusst und wieder abrufbar.

Grenze: Praktische Memory-Infrastruktur, keine Behauptung menschenähnlichen Bewusstseins.

Memory, Vault, CASS und Smart Memory MCP - Architektur

Ich zeige die Architektur bewusst als Ablauf, weil der Wert in der kontrollierten Sequenz liegt.

- CASS trennt episodische Rohdaten, Arbeitsgedächtnis und prozedurale Regeln.
- Memory Broker verteilt wichtige Fakten und markiert leichte Widersprüche.
- Vault Watcher denkt Dateiaenderungen, inkrementelles Indexing, Graph Updates und Staleness.
- Smart Memory MCP macht Lernen, Recall, Stats, Muster und Vorschläge als lokale Tools verfügbar.

Memory, Vault, CASS und Smart Memory MCP - was ich zeigen kann

- Ich habe Memory als Governance-Problem behandelt, nicht als Notizablage.
- Ich habe Rohdaten, Zusammenfassungen und Regeln getrennt.
- Ich habe Widerspruch, Veralterung und Herkunft als zentrale Risiken erkannt.
- Agenten werden gefährlich oder nutzlos, wenn sie veraltete Zusammenfassungen für Wahrheit halten.
- Widersprüche sollen erhalten und sichtbar gemacht werden, nicht still überschrieben.
- Retrieval braucht mehr als Textähnlichkeit: Nützlichkeit, Zeit und Vertrauen zählen.

Memory, Vault, CASS und Smart Memory MCP - nächste Härtung

Die öffentliche Version bleibt kuratiert und sicher: keine privaten Pfade, keine Zugangsdaten, keine rohen Arbeitsordner.

- Synthetische Public-Memory-Demo bauen.
- Retrieval und Widerspruchserkennung auf nicht-privaten Daten benchmarken.
- Memory-Provenance in Agentenantworten anzeigen.

Voice Interface and Spoken Tool Use

Warum ich es gebaut habe: Ich wollte mit meinem System sprechen können, aber Sprache ist für Tools riskant: sie ist schnell, ungenau und kann trotzdem Aktionen auslösen. Deshalb brauchte ich Muting, Tool-Grenzen, Fallbacks und Schutz gegen gefährliche Befehle.

Grenze: Terminal/Prototyp und Planungsstand sauber trennen; Wake-Word/macOS-Teile nur behaupten, wenn frisch getestet.

Voice Interface and Spoken Tool Use - Architektur

Ich zeige die Architektur bewusst als Ablauf, weil der Wert in der kontrollierten Sequenz liegt.

- Python streamt Mikrofon-Audio in Gemini Live.
- Tools erlauben Vault Search/Read/Write, Deep Thinking, Brain Status und begrenzte Shell.
- Gefährliche Shell-Muster werden blockiert, Ausführung bekommt Timeouts.
- Spätere Planung umfasst Wake Word, lokale STT, Hybrid-TTS, Barge-in und Privacy-Grenzen.

Voice Interface and Spoken Tool Use - was ich zeigen kann

- Ich habe Voice als tool-fähige Kontrolloberfläche gebaut, nicht nur als Chat-Spielerei.
- Ich habe Datenschutz, Bestätigung und Unterbrechbarkeit von Anfang an mitgedacht.
- Ich habe hochwertige Stimmen mit praktischer Tool-Nutzung verbunden.
- Voice senkt Reibung beim Auslösen und muss deshalb Reibung beim Risiko erhöhen.
- Barge-in ist ein Sicherheitsfeature, weil ich das System sofort stoppen können muss.
- Eine schöne Stimme ist wertlos, wenn Autorität und Privacy unklar bleiben.

Voice Interface and Spoken Tool Use - nächste Härtung

Die öffentliche Version bleibt kuratiert und sicher: keine privaten Pfade, keine Zugangsdaten, keine rohen Arbeitsordner.

- Öffentliche Demo ohne private Mikrofon-/Vault-Daten bauen.
- Safe Tools und confirmation-required Tools klar trennen.
- Transkript-Audit und Korrekturpfade ergänzen.

ANIMA Experimental Kernel

Warum ich es gebaut habe: Ich wollte spekulative Ideen über Kognition nicht als große Behauptung stehen lassen, sondern in Code, Tests, Metriken und Evidenzgrenzen übersetzen.

Grenze: Kein Beweis für künstliches Bewusstsein; ein ambitioniertes, getestetes Experimentierfeld.

ANIMA Experimental Kernel - Architektur

Ich zeige die Architektur bewusst als Ablauf, weil der Wert in der kontrollierten Sequenz liegt.

- Kernel-, State- und Typmodule bilden den Kern.
- Primitiven wie Qualia, Engram, Valence, Nexus, Impulse, Trace, Mirror und Flux werden testbar gemacht.
- Temporal- und Metrikschichten erlauben Messung statt reiner Philosophie.
- Benchmark- und Evidence-Status-Dateien verhindern überzogene Claims.

ANIMA Experimental Kernel - was ich zeigen kann

- Ich kann eine ambitionierte Idee in ein testbares Paket verwandeln.
- 446 lokale Tests wurden im Rebuild erfolgreich ausgeführt.
- Ich begrenze die öffentliche Aussage bewusst: Architektur/Testbed, nicht Bewusstseinsbeweis.
- Große Begriffe brauchen mehr Evidenz als normale Softwareclaims.
- Nicht-signifikante Ergebnisse können wertvoll sein, wenn sie die nächste Hypothese formen.
- Die glaubwürdige Version ist die begrenzte Version.

ANIMA Experimental Kernel - nächste Härtung

Die öffentliche Version bleibt kuratiert und sicher: keine privaten Pfade, keine Zugangsdaten, keine rohen Arbeitsordner.

- Benchmarkbericht mit Baselines und Statistik sauber öffentlich machen.
- Philosophie, Architektur und gemessene Resultate klar trennen.
- ANIMA als Experimentmodul nutzen, nicht als Hauptclaim.